

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

Тема: UI-тестирование

Студент гр. 4341

Студент гр. 4341

Студент гр. 4343

Студент гр. 4345

Руководитель

Д.Д. Скрипникова

В.Г. Филатов

Р.Б. Едыгеев

Н.С. Лунченко

А.М. Шевелёва

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: Д. Д. Скрипникова

Группа: 4341

Тема практики: UI-тестирование

Задание на практику:

Разработать набор автоматизированных UI-тестов для веб-приложения на языке Java с использованием технологий Maven, JUnit 5, Selenide и паттерна Page Object Model.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 00.00.0000

Дата защиты отчета: 00.00.0000

Студент	_____	Д.Д. Скрипникова
---------	-------	------------------

Руководитель	_____	А.М. Шевелёва
--------------	-------	---------------

АННОТАЦИЯ

Отчёт по летней учебной практике содержит описание разработанного набора автоматизированных UI-тестов для веб-приложения GitHub.

В рамках работы проведён анализ предметной области, обоснован выбор технологического стека: Java 17, Maven, JUnit 5, Selenide. Описана архитектура тестового проекта, построенная на основе паттерна Page Object Model.

Разработано 8 автоматизированных тестов, покрывающих создание публичного и приватного репозитория, создание и удаление файлов, создание и удаление веток, создание и закрытие Pull Request'ов.

В отчёте приведены описание архитектуры проекта с UML-диаграммой, а также изложены результаты выполнения работы.

SUMMARY

The Summer internship report contains a description of the developed set of automated UI tests for the GitHub web application.

As part of the work, an analysis of the subject area was carried out, and the choice of a technology stack was justified: Java 17, Maven, JUnit 5, Selenide. The architecture of the test project based on the Page Object Model pattern is described.

8 automated tests have been developed covering the creation of public and private repositories, the creation and deletion of files, the creation and deletion of branches, the creation and closure of Pull Requests.

The report describes the architecture of the project with a UML diagram, as well as the results of the work.

СОДЕРЖАНИЕ

Для составления Содержания удобно использовать автоматически собираемое Оглавление - если вы корректно использовали соответствующие стили заголовков (согласно данному шаблону), то в Содержании автоматически войдут нужные разделы с номерами страниц (разделы Задание, Календарный план, Реферат, Abstract в Содержании не указываются).

Для заголовков первого уровня (названия глав) используйте стиль «Заголовок 2», для разделов - «Заголовок 3», для подразделов следующих уровней - «Заголовок 4».

ВВЕДЕНИЕ	7
1 ПОСТАНОВКА ЗАДАЧИ	8
1.1 Создание публичного репозитория	8
1.2 Создание приватного репозитория	9
1.3 Создание файла	9
1.4 Удаление файла	11
1.5 Создание ветки	13
1.6 Удаление ветки	15
1.7 Создание Pull Request	16
1.8 Заккрытие Pull Request	19
2 РЕЗУЛЬТАТЫ РАБОТЫ	21
2.1. Формулировка задачи 1	21
2.2. Формулировка задачи 2	21
2.3. Формулировка задачи 3	21
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	22
3.1 Базовый класс BaseElement	22
3.2 Какие элементы были созданы	22
3.3 Составные элементы	23
3.4 Логирование	23
4 ОТЗЫВ РУКОВОДИТЕЛЯ	24

ЗАКЛЮЧЕНИЕ	25
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	26
ПРИЛОЖЕНИЕ А (заголовок второго уровня)	27

ВВЕДЕНИЕ

В рамках летней учебной практики по направлению «UI-тестирование» перед командой стояла задача разработать набор автоматизированных UI-тестов для веб-приложения на языке Java с использованием технологий Maven, JUnit 5, Selenide и паттерна Page Object Model.

Объект тестирования: веб-платформа GitHub.

Предмет тестирования: функциональный блок «Управление репозиториями» (создание, удаление репозитория, работа с файлами, ветками, Pull Request'ами).

Вклад каждого участника:

- Скрипникова Дарья: разработка элементов страниц, создание иерархии базовых и составных элементов, организация структуры проекта, настройка логирования
- Лунченко Никита: реализация страниц авторизации, управления репозиториями и файлами, интеграция элементов в страницы.
- Филатов Вячеслав и Едыгеев Руслан: разработка тестов, написание и отладка тестов для проверки функциональности работы с репозиториями, файлами, ветками и Pull Request'ами.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Создание публичного репозитория

Цель: проверить возможность создания нового репозитория с публичной видимостью.

Ожидаемый результат: репозиторий создаётся, отображается в списке репозиториях пользователя и имеет статус «Public».

На рисунках 1-2 (рис 1 - рис 2) представлены шаги выполнения теста.

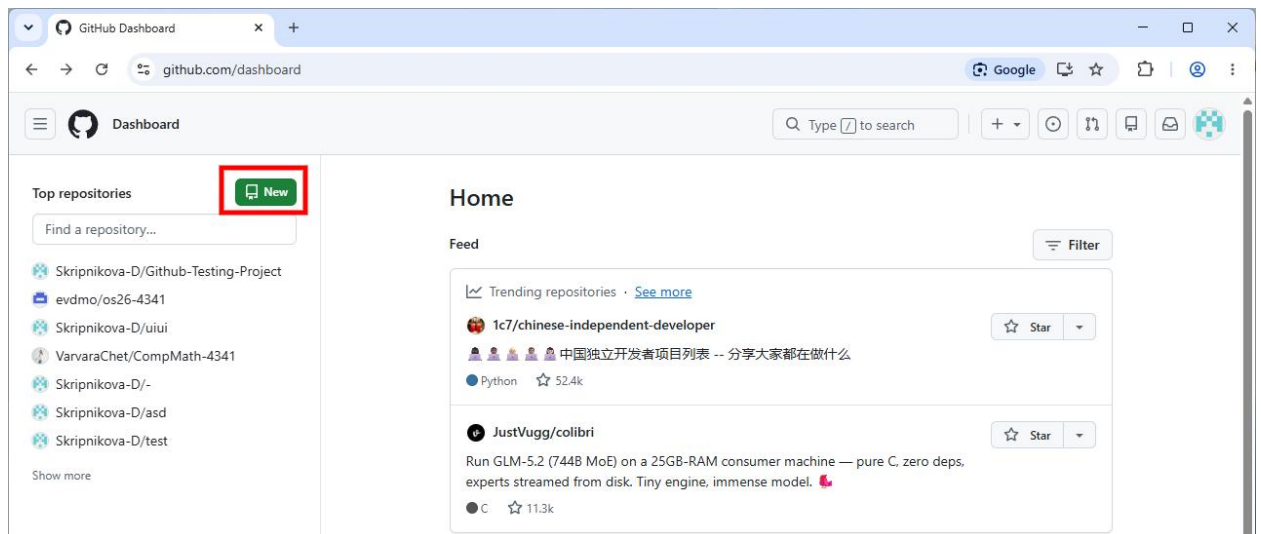


Рисунок 1 – Создание нового репозитория. Часть 1

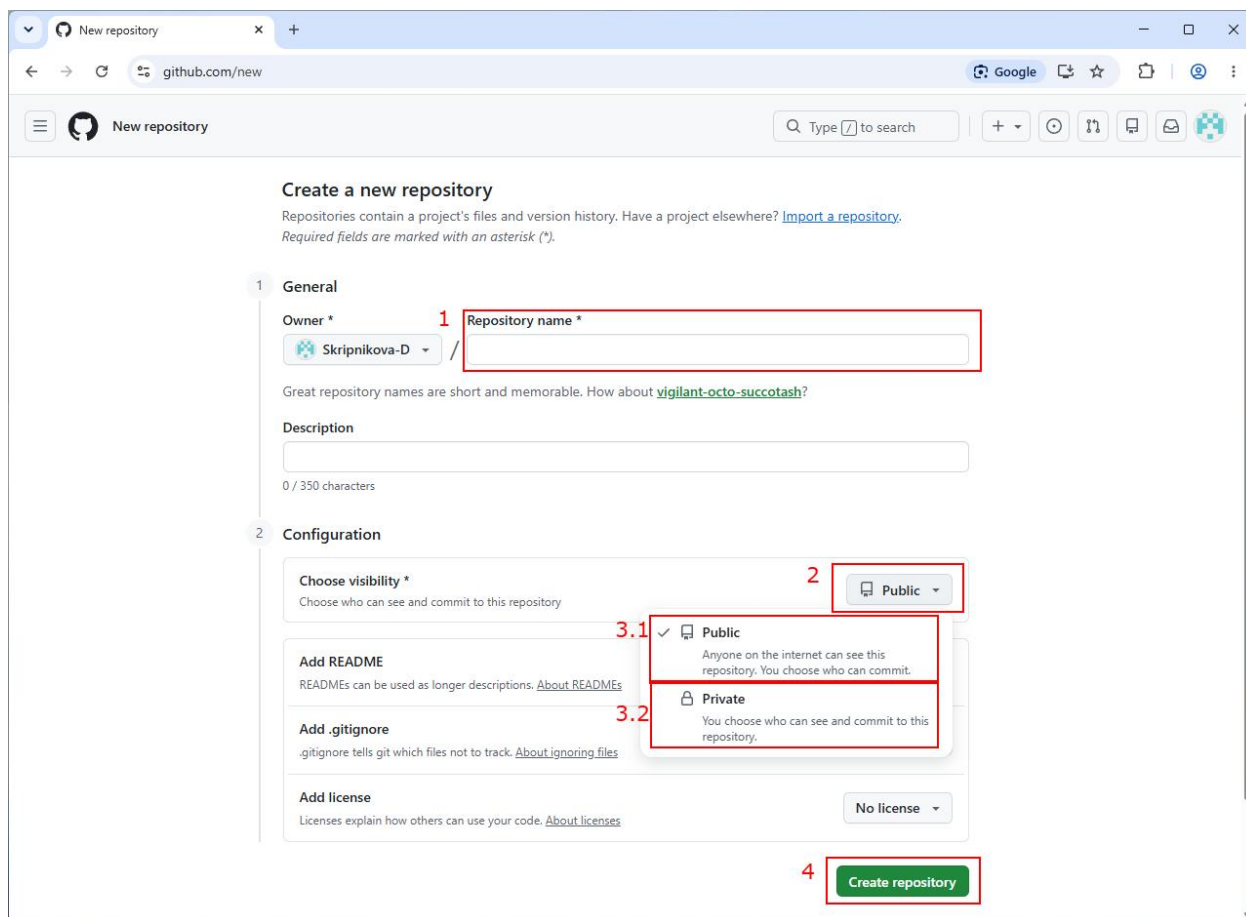


Рисунок 2 – Создание нового репозитория. Часть 2

1.2 Создание приватного репозитория

Цель: проверить возможность создания нового репозитория с приватной видимостью.

Ожидаемый результат: репозиторий создаётся, отображается в списке репозиториях пользователя и имеет статус «Private».

Шаги выполнения теста показаны на рисунках 1 - 2 (см. выше) с поправкой на то, что для создания приватного репозитория необходимо следовать шагу 3.2 (рис 2), а не 3.1

1.3 Создание файла

Цель: проверить возможность создания нового файла через веб-интерфейс GitHub.

Ожидаемый результат: файл создаётся и отображается в списке файлов репозитория.

Шаги выполнения теста показаны на рисунках 3 - 5 (рис 3 - рис 5)

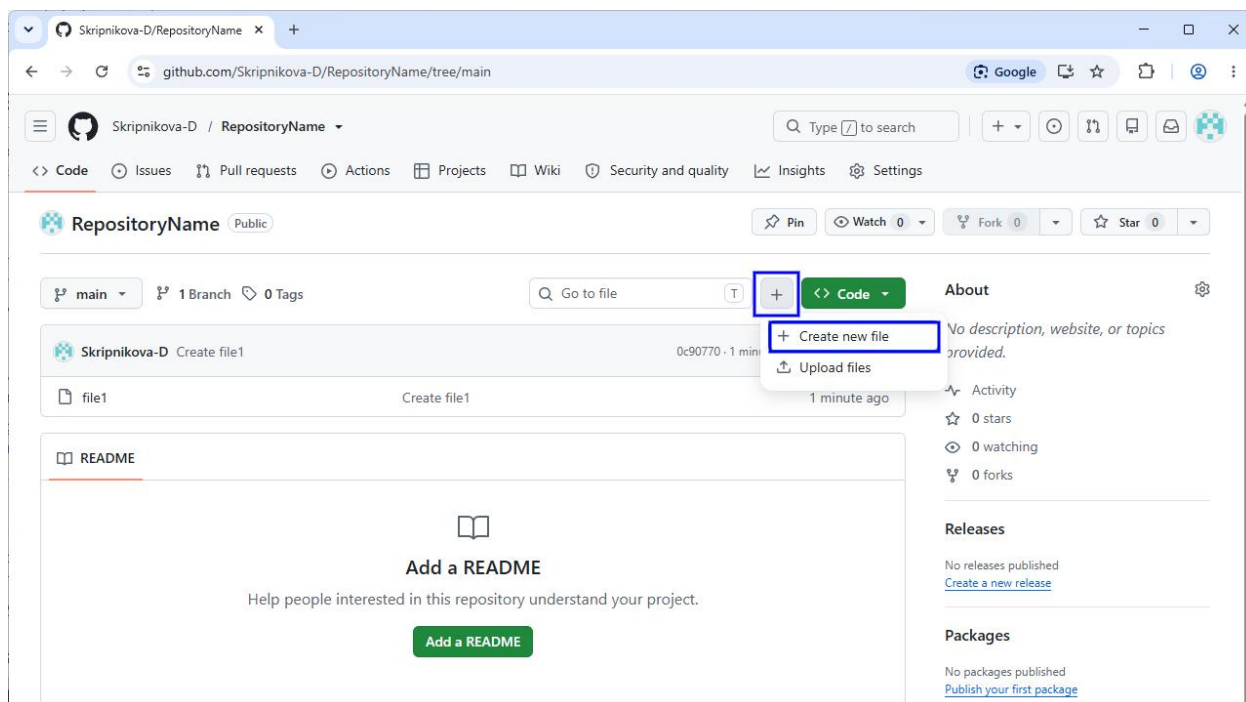


Рисунок 3 – Создание нового файла. Часть 1

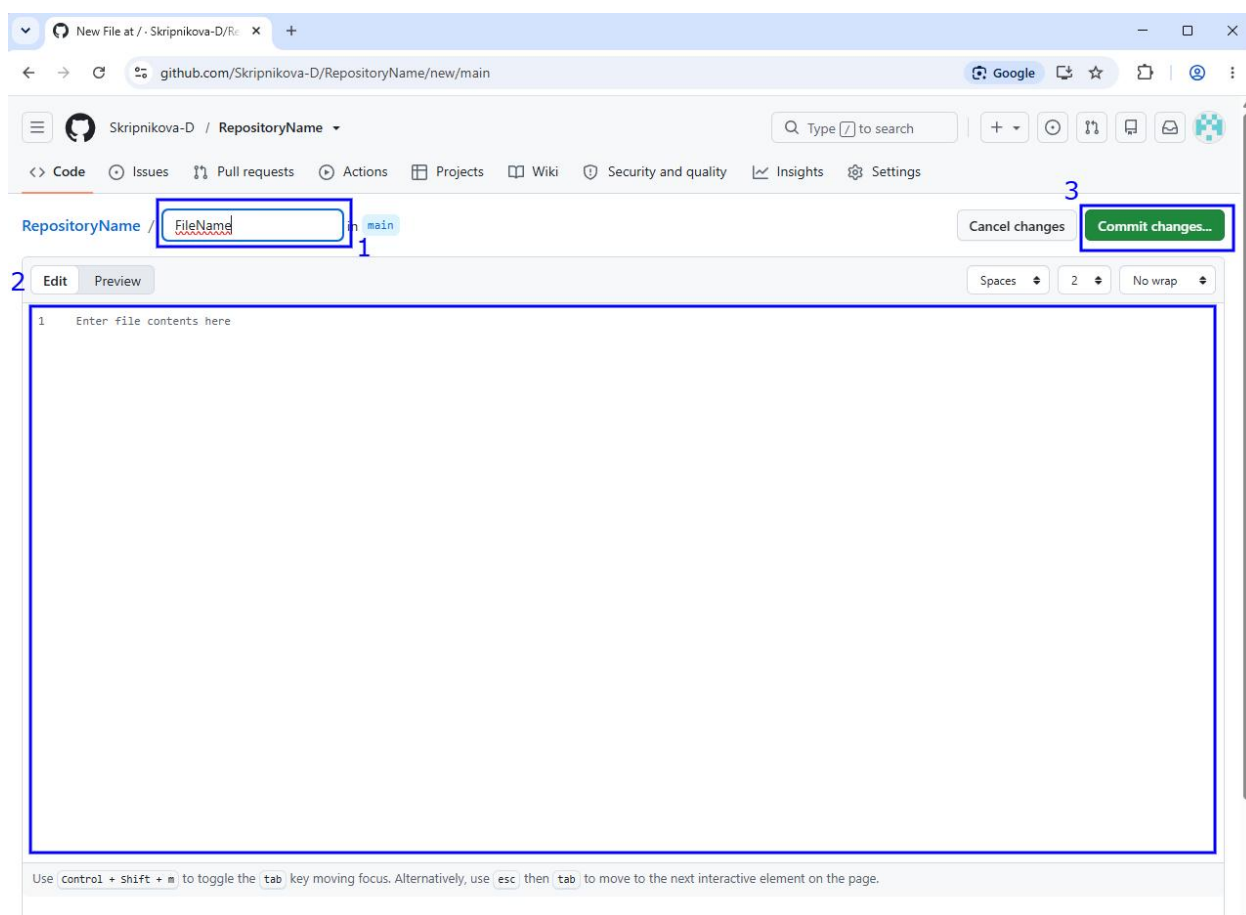


Рисунок 4 – Создание нового файла. Часть 2

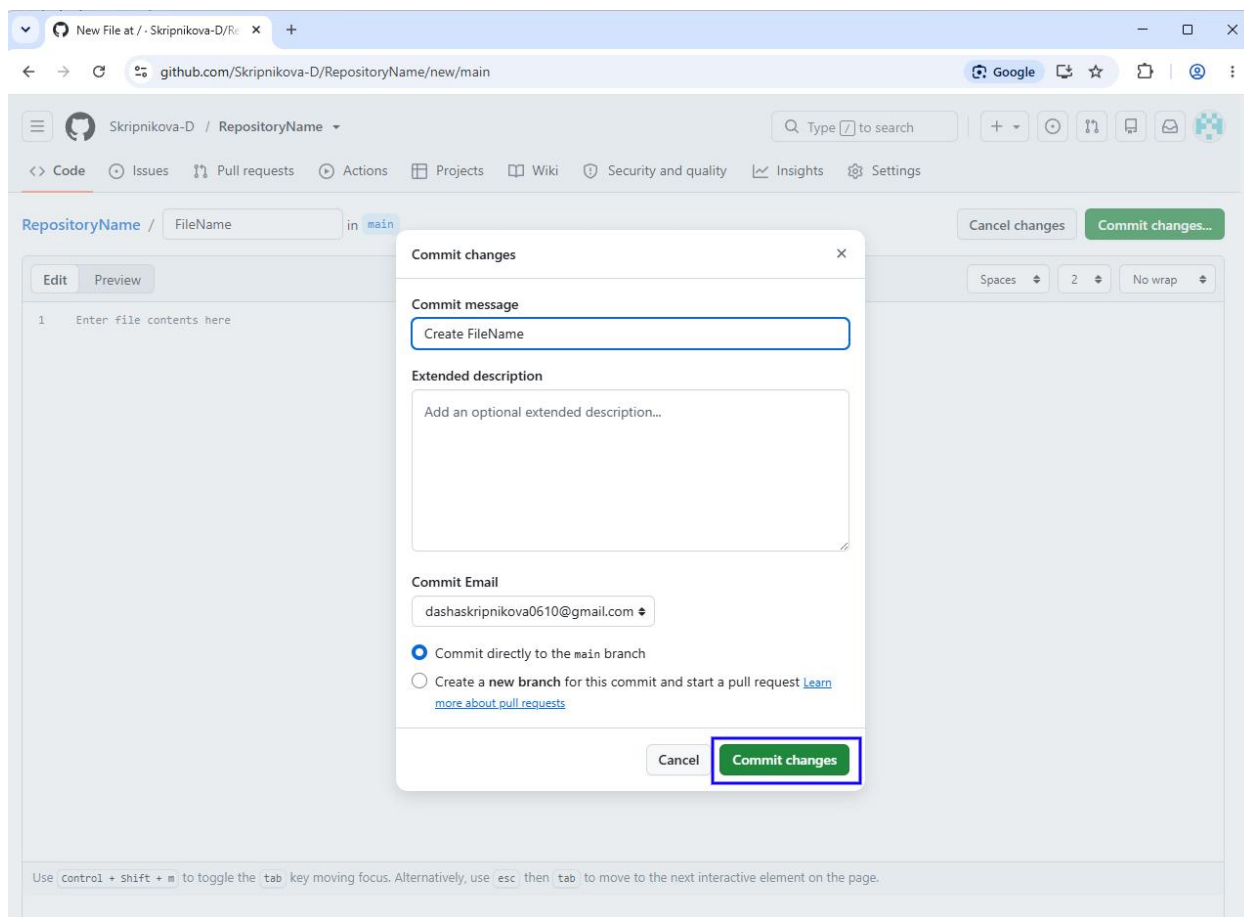


Рисунок 5 – Создание нового файла. Часть 3

1.4 Удаление файла

Цель: проверить возможность удаления существующего файла.

Ожидаемый результат: файл удаляется и не отображается в списке файлов.

Шаги выполнения теста показаны на рисунках 6 - 9 (рис 6 - рис 9)

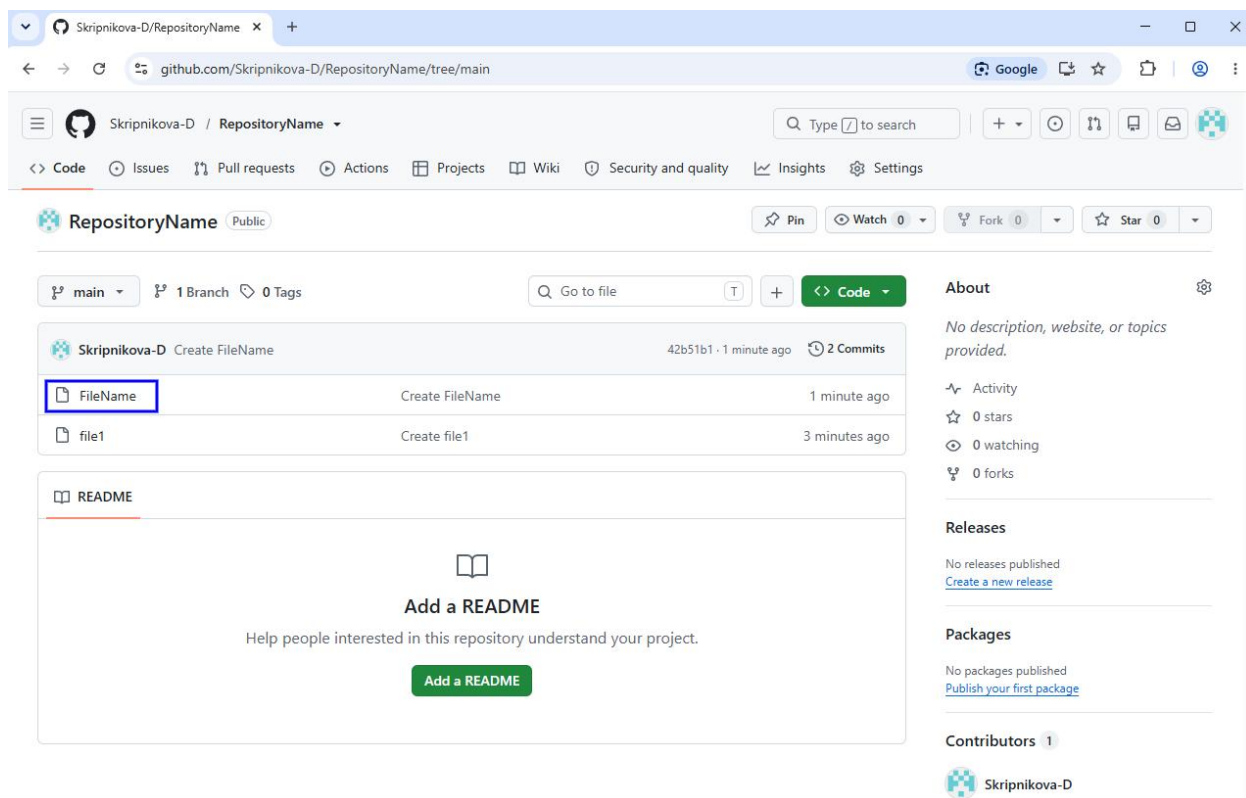


Рисунок 6 – Удаление файла. Часть 1

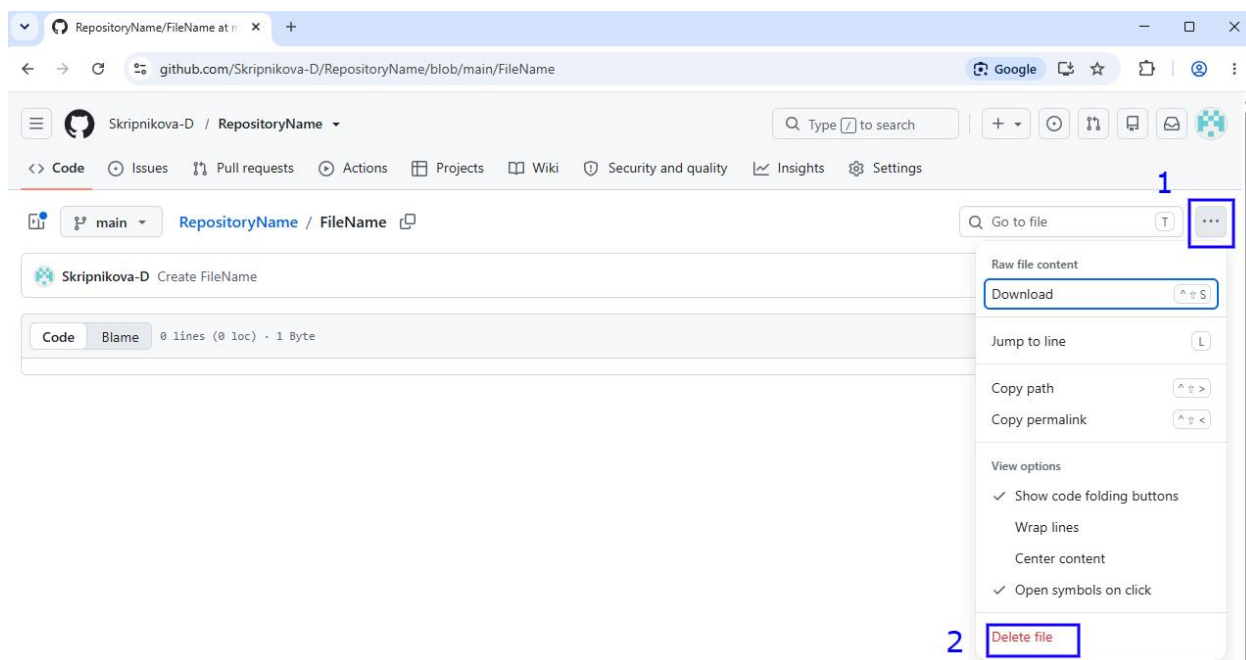


Рисунок 7 – Удаление файла. Часть 2

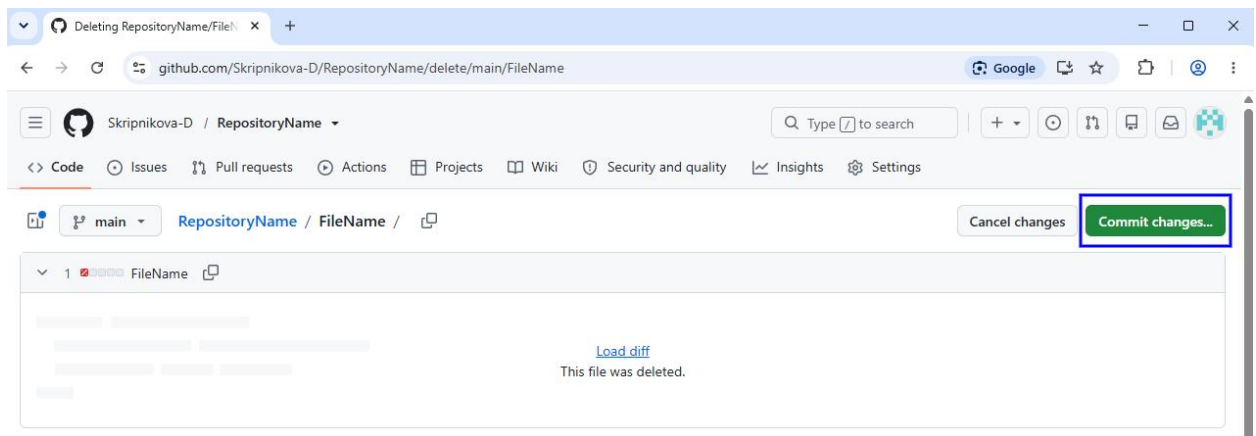


Рисунок 8 – Удаление файла. Часть 3

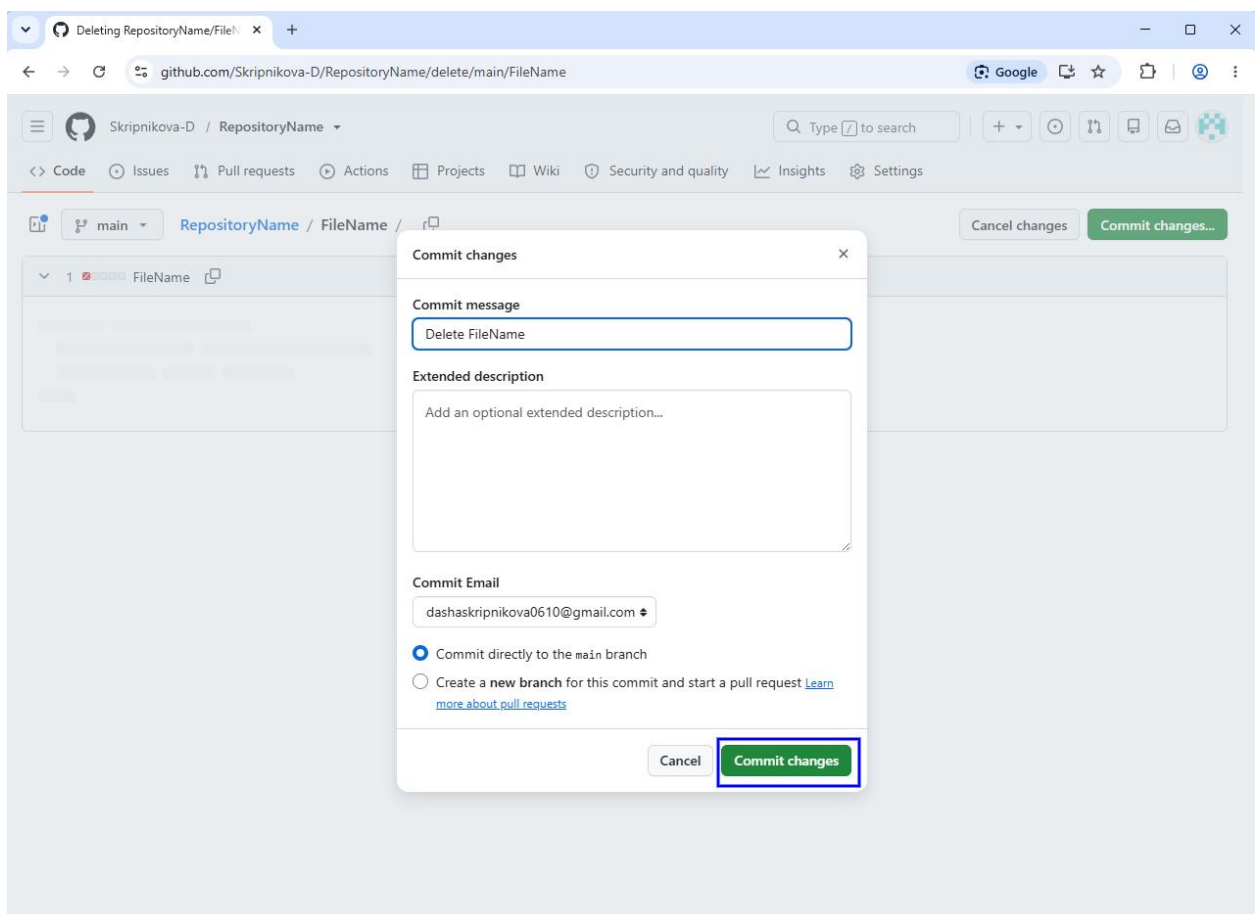


Рисунок 9 – Удаление файла. Часть 4

1.5 Создание ветки

Цель: проверить возможность создания новой ветки в репозитории.

Ожидаемый результат: новая ветка успешно создаётся и отображается в списке веток репозитория.

Шаги выполнения теста показаны на рисунках 10 - 12 (рис 10 - рис 12)

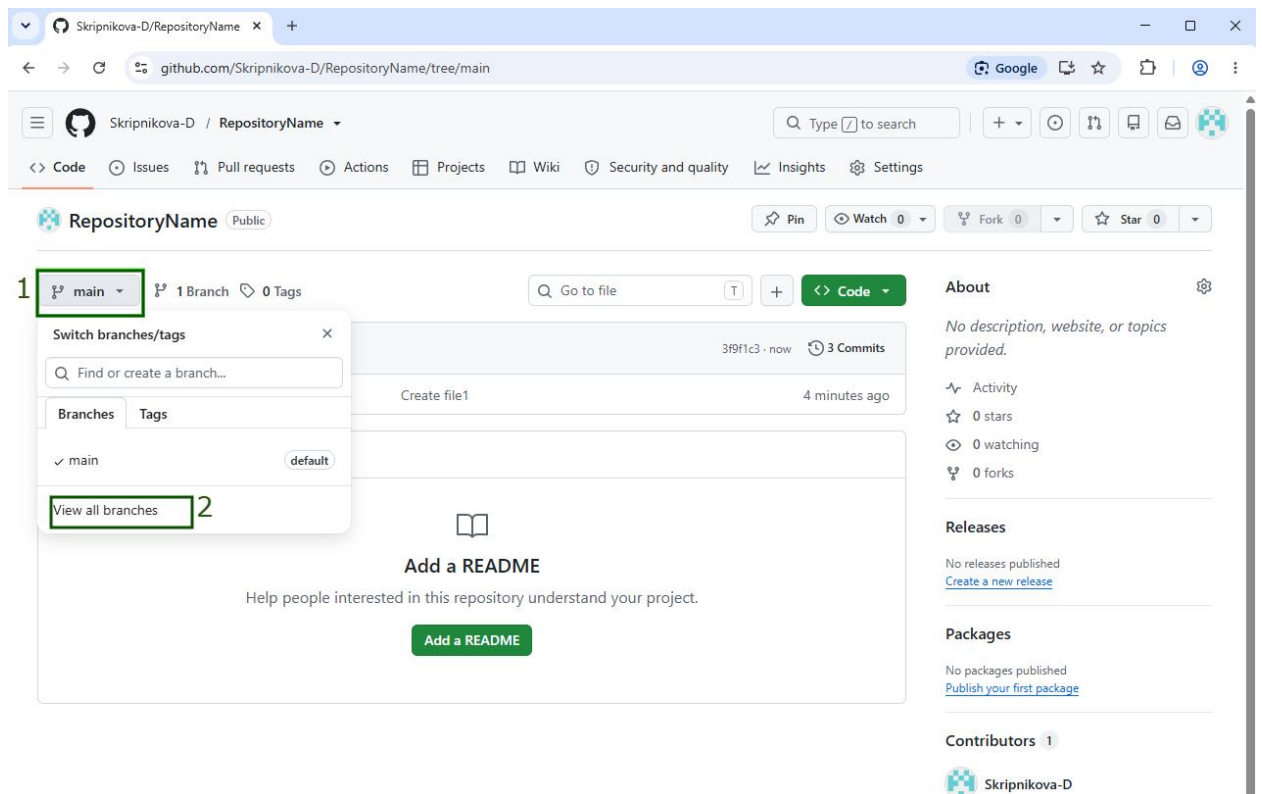


Рисунок 10 – Создание ветки. Часть 1

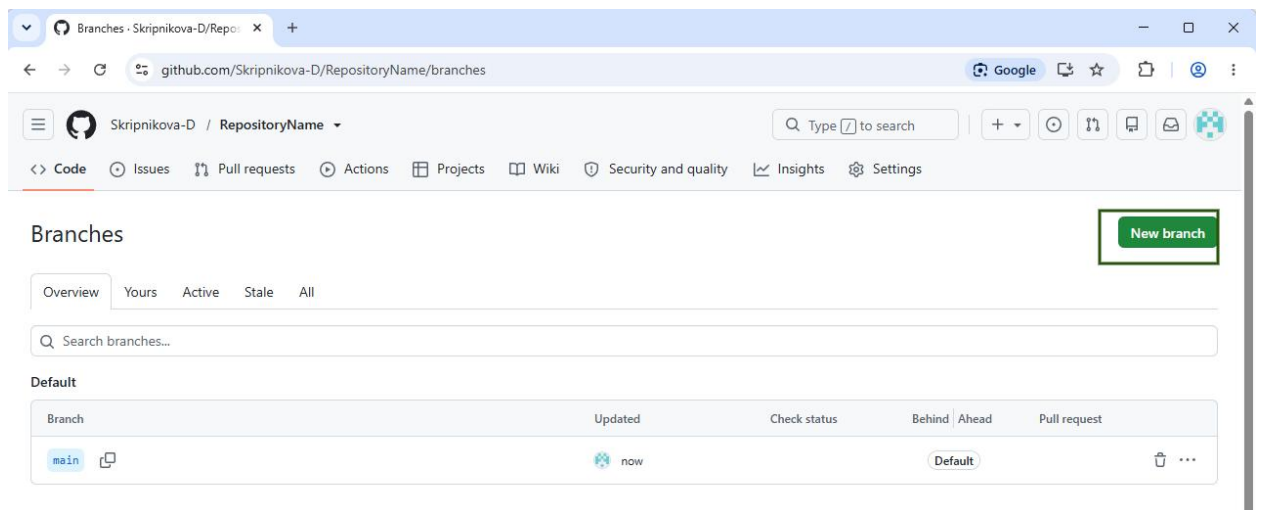


Рисунок 11 – Создание ветки. Часть 2

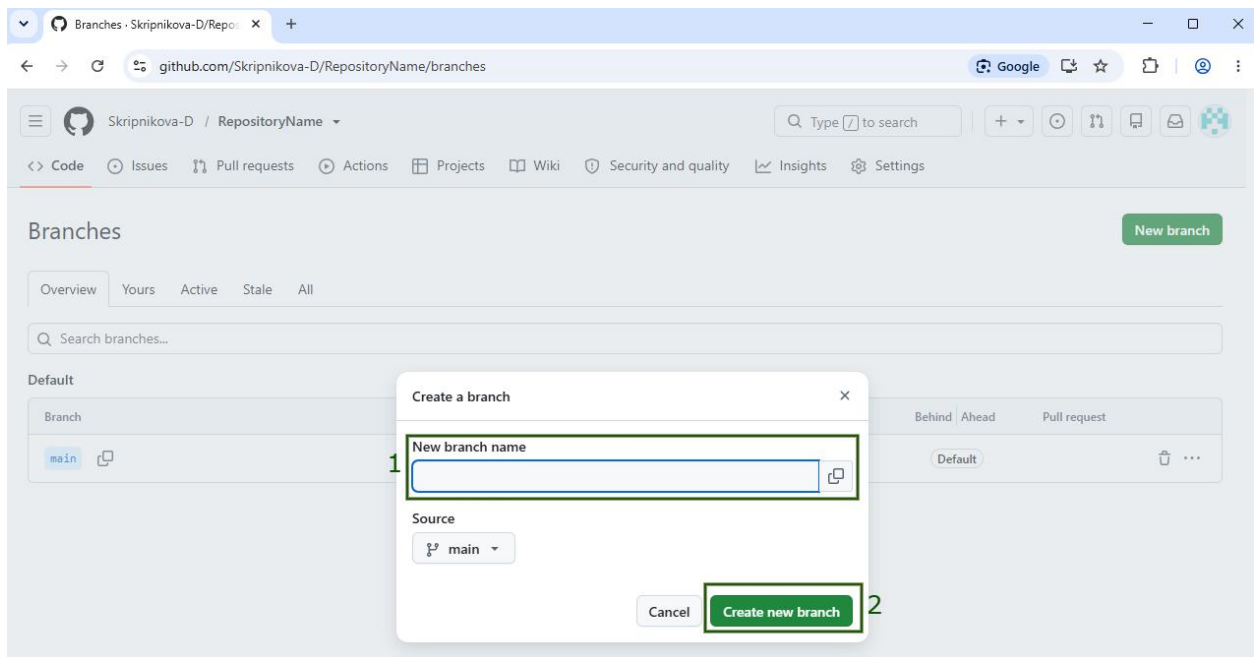


Рисунок 12 – Создание ветки. Часть 3

1.6 Удаление ветки

Цель: проверить возможность удаления ветки.

Ожидаемый результат: ветка удаляется и не отображается в списке веток.

Шаги выполнения теста показаны на рисунках 13 - 14 (рис 13 - 14 рис)

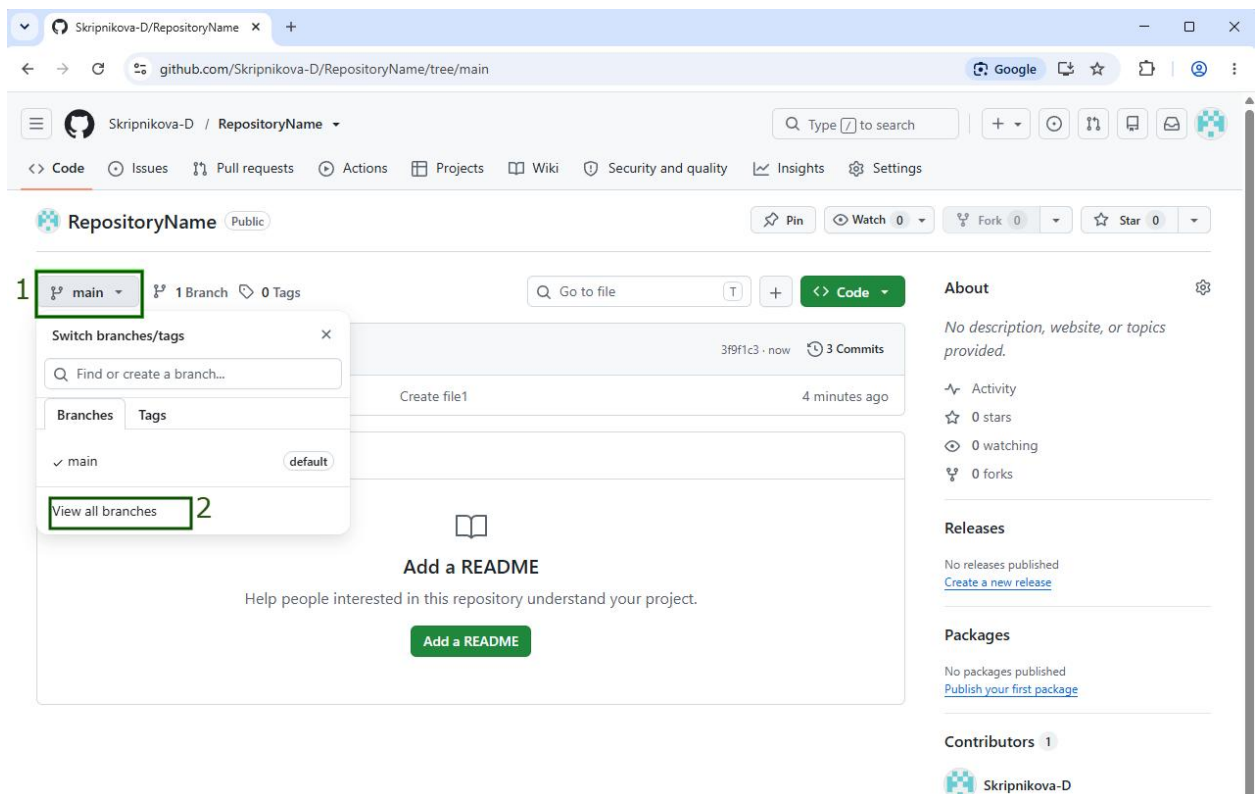


Рисунок 13 – Удаление ветки. Часть 1

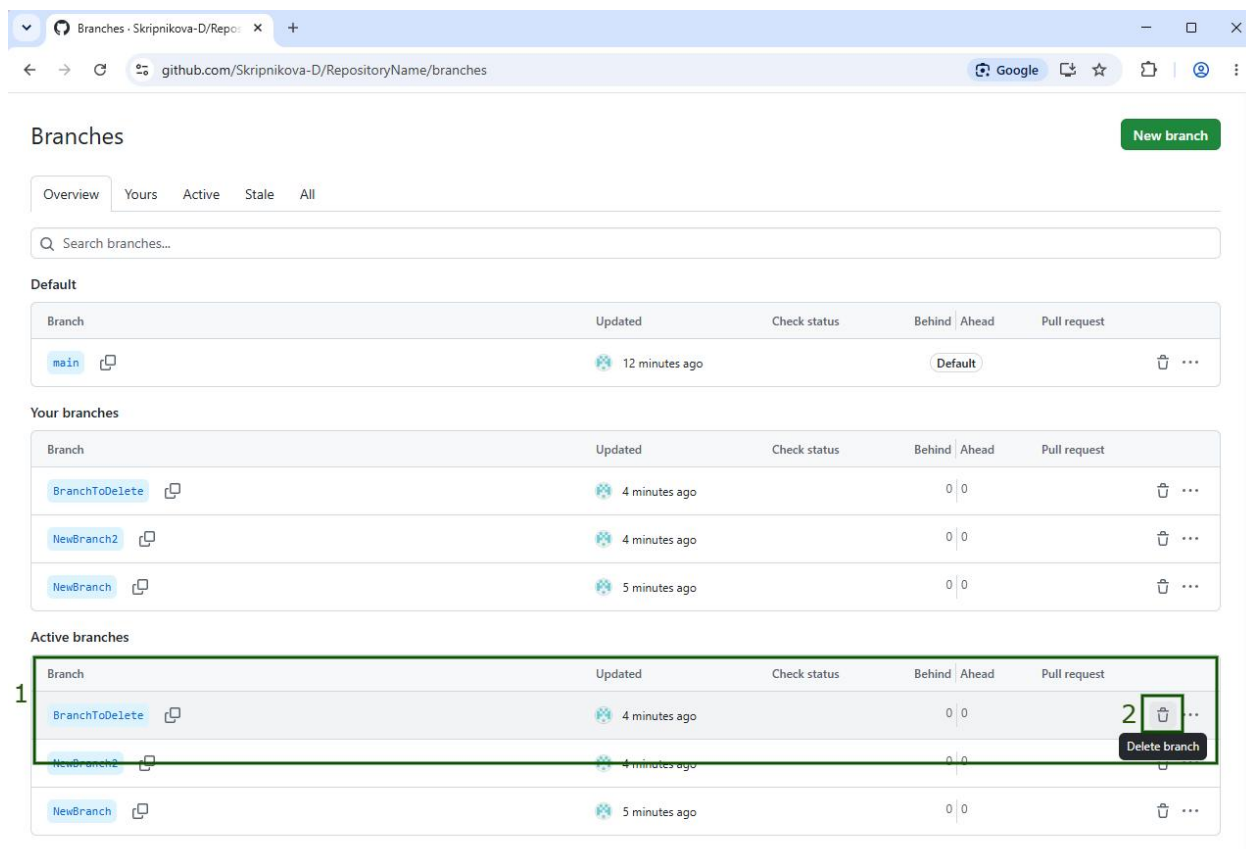


Рисунок 14 – Удаление ветки. Часть 2

1.7 Создание Pull Request

Цель: проверить процесс создания Pull Request для слияния изменений из одной ветки в другую.

Ожидаемый результат: Pull Request создаётся, открыт и отображается в списке Pull Request'ов.

Шаги выполнения теста показаны на рисунках 15 - 19 (рис 15 - 19 рис)

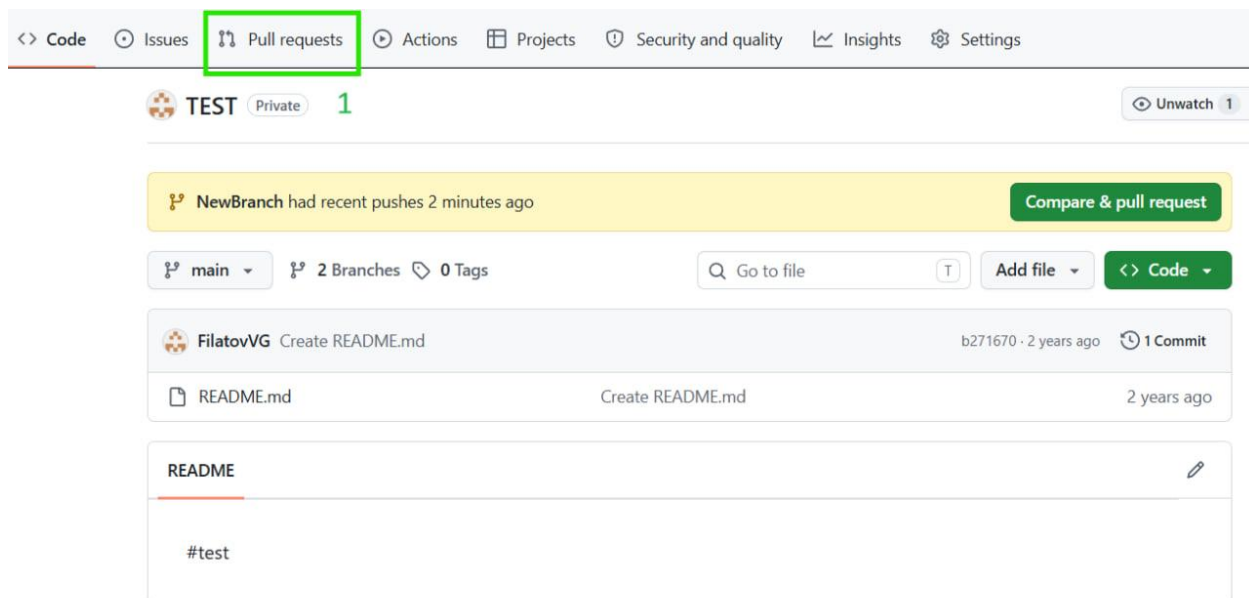


Рисунок 15 – Создание Pull Request. Часть 1

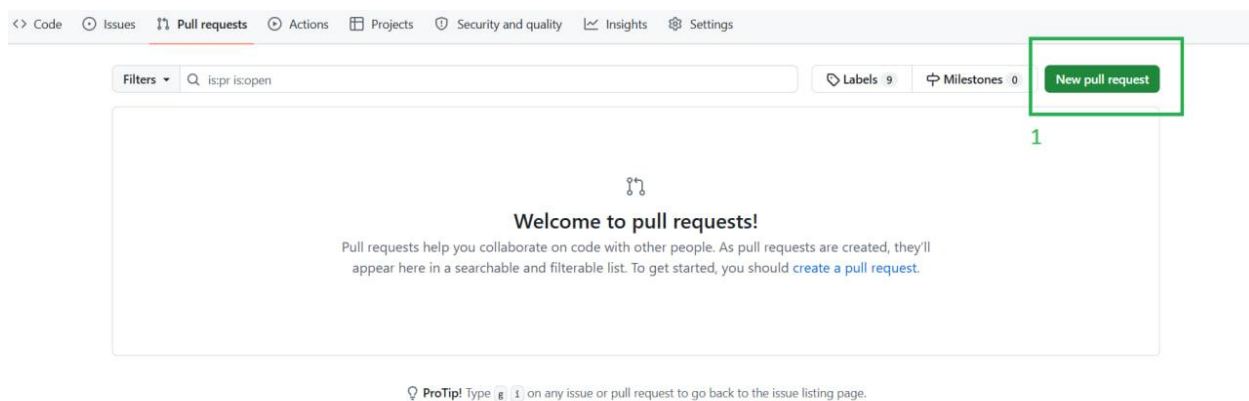


Рисунок 16 – Создание Pull Request. Часть 2

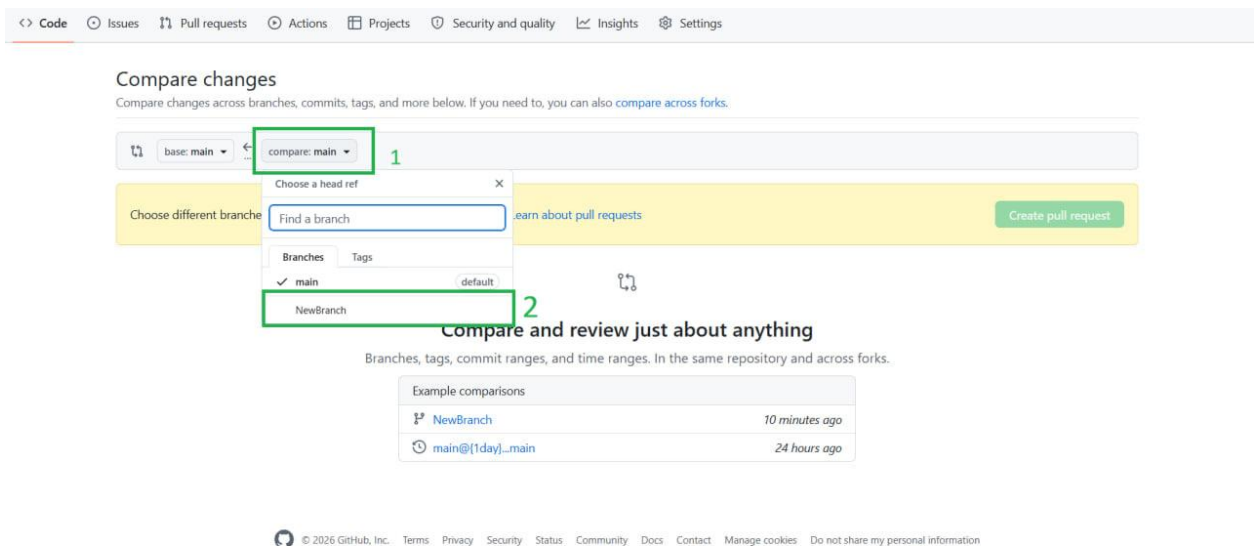


Рисунок 17 – Создание Pull Request. Часть 3

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

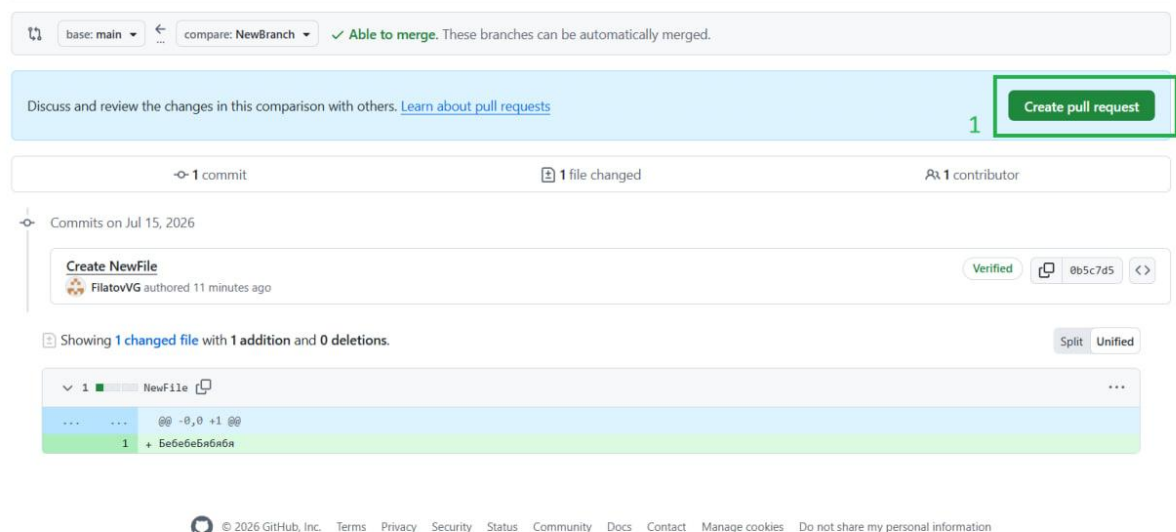


Рисунок 18 – Создание Pull Request. Часть 4

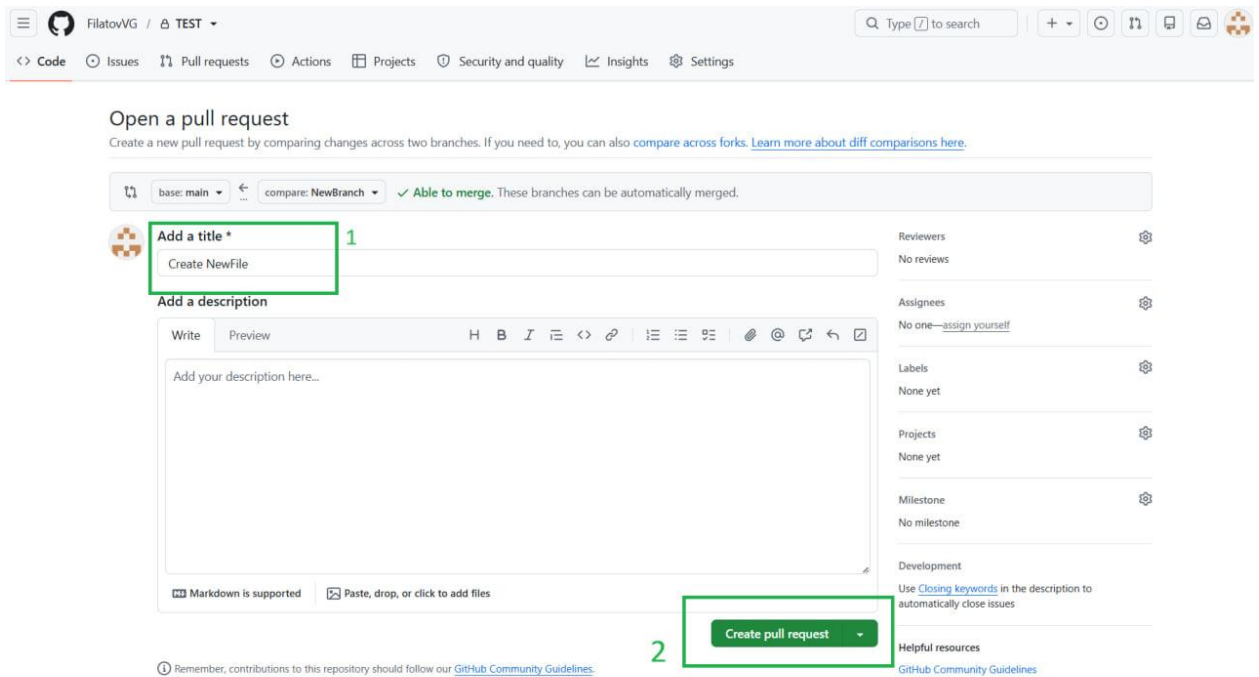


Рисунок 19 – Создание Pull Request. Часть 5

1.8 Заккрытие Pull Request

Цель: проверить возможность закрытия Pull Request без слияния.

Ожидаемый результат: Pull Request закрывается и приобретает статус «Closed».

Шаги выполнения теста показаны на рисунках 20 - 22 (рис 20 - 22 рис)

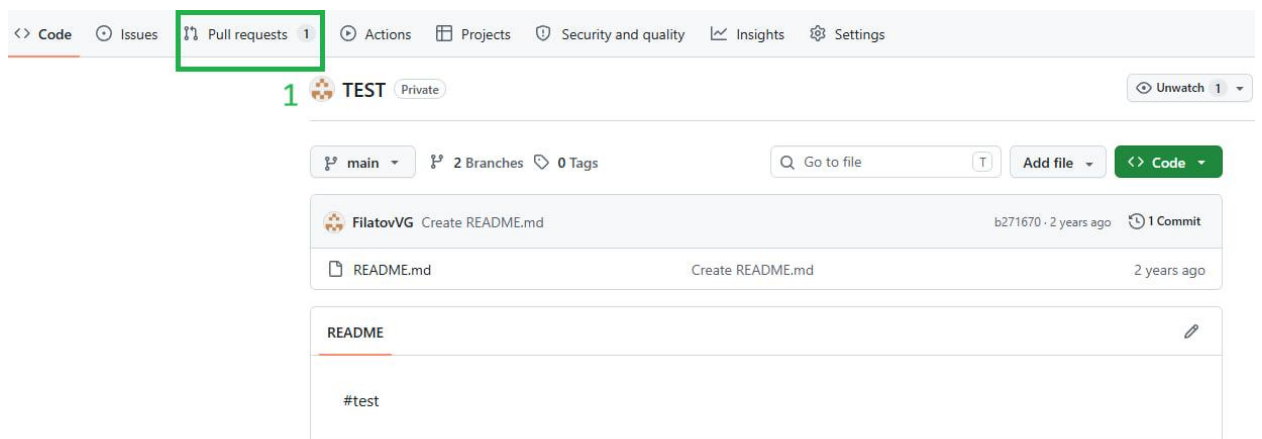


Рисунок 20 – Заккрытие Pull Request. Часть 1

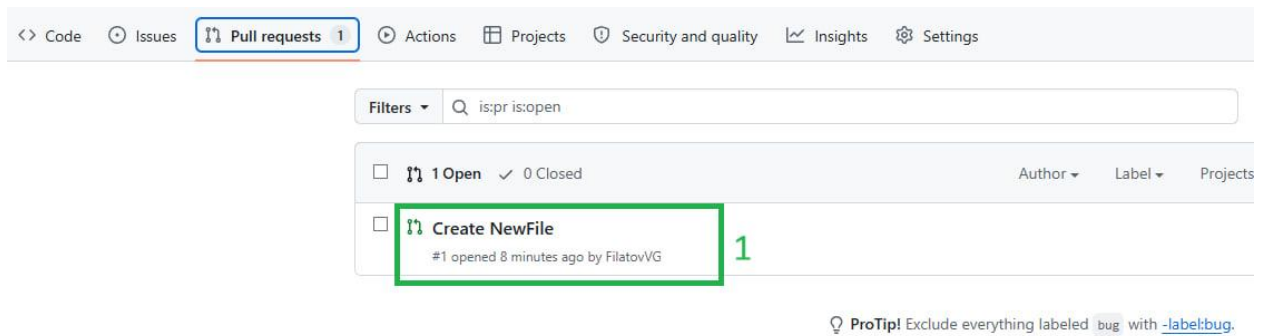


Рисунок 21 – Заккрытие Pull Request. Часть 2

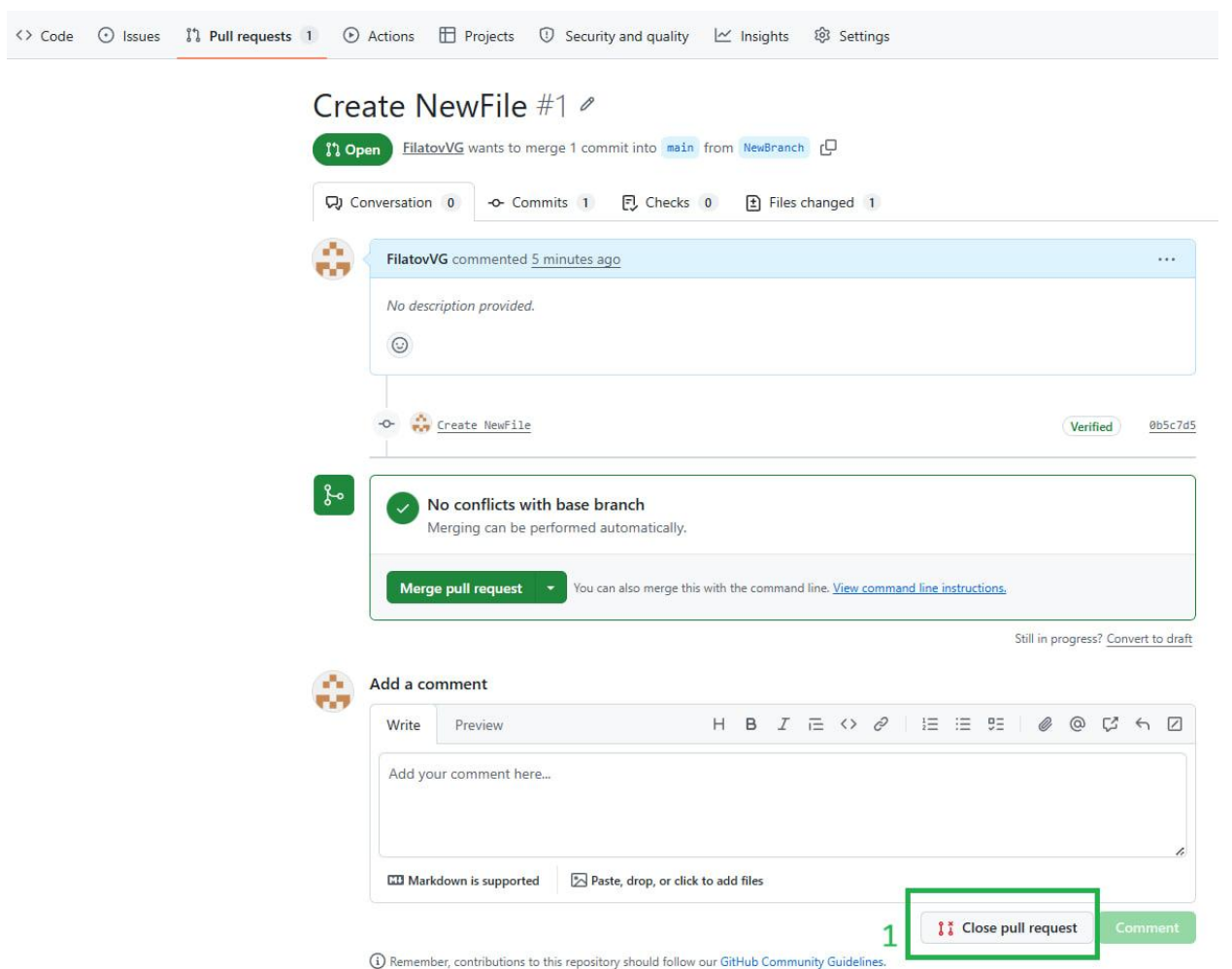


Рисунок 22 – Заккрытие Pull Request. Часть 3

2 РЕЗУЛЬТАТЫ РАБОТЫ

Данная глава должна содержать описание процесса решения и результаты по каждой задаче практики из предыдущего раздела. По приведённому тексту читатель должен понять, *что* и *как* было сделано для выполнения поставленных задач.

Повествование ведется обезличенно.

2.1. Формулировка задачи 1 (Элементы)

Описание процесса решения задачи №1, реализованные или использованные классы / модули / сервисы и т.д.

2.2. Формулировка задачи 2 (Страницы)

...

2.3. Формулировка задачи 3 (Тесты)

...

во втором разделе краткое описание классов и методов + uml диаграмма (эта часть будет одинакова для всех)

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В рамках проекта моей задачей была разработка элементов страниц. Элементы - это базовые части интерфейса: кнопки, поля ввода, ссылки и другие компоненты. Каждый такой класс отвечает за поиск нужного элемента на странице и выполнение простых действий, например, клик или ввод текста.

Такой подход позволяет:

- Не писать один и тот же код поиска по несколько раз
- Быстро менять XPath в одном месте, если структура страницы изменилась
- Упростить написание страниц и тестов

3.1 Базовый класс BaseElement

Был создан базовый класс BaseElement, от которого наследуются все остальные элементы. Он делает общую работу:

- Поиск элемента - по шаблону XPath или по готовому XPath
- Ожидание - автоматически ждёт, пока элемент появится на странице (до 5 секунд)
- Проверка видимости - метод isDisplayed() возвращает true, если элемент виден на странице.

Благодаря этому классу не нужно писать одинаковый код для каждого нового элемента.

3.2 Какие элементы были созданы

Button - для кнопок. Поиск осуществляется:

- по ID
- по тексту на кнопке
- по атрибуту aria-label
- по атрибуту data-testid
- по классу.

Input - для полей ввода. Поиск осуществляется:

- по ID

- по атрибуту aria-label
- по атрибуту name
- по тексту подписи (label).

Link - для ссылок. Поиск осуществляется:

- по адресу (href)
- по тексту ссылки
- по атрибуту aria-label
- по адресу с дополнительным уточнением.

Summary - для раскрывающихся списков. Поиск по тексту или aria-label.

DropDownItem - для пунктов выпадающих списков. Поиск по роли меню и тексту пункта.

CodeMirrorInput - для редактора кода. Особенность: перед вводом текста нужно сначала кликнуть по полю, чтобы его активировать.

3.3 Составные элементы

Для некоторых элементов потребовались отдельные классы, так как они состоят из нескольких частей.

BranchTable - для таблицы с ветками. Класс находит таблицу по заголовку «Active branches», находит нужную строку по названию ветки и удаляет её.

SelectMenu - для выбора веток при создании Pull Request. Класс находит меню по заголовку («Choose a base ref» или «Choose a head ref») и выбирает ветку по названию.

3.4 Логирование

В проекте настроено автоматическое логирование через Log4j. Теперь все действия Selenium (открытие страниц, клики, ввод текста) записываются в логи сами, без необходимости писать logger.info() в каждом методе.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

Если прохождение практики было по кафедральным темам достаточно добавить формулировку:

По результатам работы учебная практика оценена на <ваша_оценка>.

Если прохождение практики было по своей теме, то необходимо добавить скан отзыва руководителя с оценкой и подписью, скан должен быть в высоком качестве на всю страницу.

Для прохождения автоматической проверки для скана необходима подпись и ссылка в тексте, т.к. он считается рисунком.

Отзыв руководителя с оценкой (см. рис. X).

Отзыв
о прохождении учебной практики

<ФИО студента>, студент(ка) группы <номер группы>, второго курса бакалавриата Санкт-Петербургского электротехнического университета “ЛЭТИ” им В.И. Ульянова, проходил(а) учебную практику по теме “<тема практики>” с <дата начала> по <дата окончания>.

В течение практики обучающийся(ая) <ФИО студента> выполнял <задание на практику>.

В результате практики обучающийся(ая) <результат практики>. Получаемую работу обучающийся(ая) <ФИО студента> выполнял <характеристика личных качеств практиканта>.

По результатам работы <ФИО студента> учебной практику оцениваю на <оценка: Отлично, Хорошо, Удовлетворительно, Неудовлетворительно>.

Подпись руководителя практики:

<Должность><дата> <подпись>/<ФИО>

Рисунок X - Отзыв руководителя

ЗАКЛЮЧЕНИЕ

В ходе выполнения летней учебной практики по направлению «UI-тестирование» была достигнута поставленная цель - разработан набор автоматизированных UI-тестов для веб-приложения (GitHub) на языке Java. В рамках работы был реализован полноценный тестовый проект, построенный на паттерне Page Object Model, что обеспечило его удобство для командной разработки.

Поставленные задачи были решены в полном объёме. Разработано 8 автоматизированных тестов, покрывающих ключевые функции выбранного блока тестирования: создание репозитория с различными настройками видимости, работу с файловой структурой (создание и удаление файлов), управление ветками (создание и удаление), а также создание и закрытие Pull Request'ов.

Значительное внимание было уделено созданию устойчивых XPath-выражений, позволяющих надёжно идентифицировать элементы на странице.

Настроенное логирование через Log4j позволяет автоматически фиксировать все действия Selenide, что упрощает отладку и анализ результатов выполнения тестов.

В ходе работы были освоены навыки командной разработки, работы с системой контроля версий Git, написания автотестов на Java. Получен практический опыт применения паттерна Page Object Model, работы с XPath и настройки логирования. Результаты работы подтверждены успешным прохождением всех тестов, что свидетельствует о корректности реализованных проверок.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Selenide [Электронный ресурс]. URL: <https://selenide.org> (дата обращения: 09.07.2026).
2. Java. Базовый курс [Электронный ресурс]. URL: <https://stepik.org/course/187/info> (дата обращения: 08.07.2026).
3. Apache Maven Project [Электронный ресурс]. URL: <https://maven.apache.org> (дата обращения: 09.07.2026).
4. JUnit [Электронный ресурс]. URL: <https://junit.org> (дата обращения: 11.07.2026).
5. Github-Testing-Project [Электронный ресурс] : летняя учебная практика 2026, UI-Тестирование GitHub. URL: <https://github.com/Skripnikova-D/Github-Testing-Project> (дата обращения: 12.07.2026).

ПРИЛОЖЕНИЕ А (ЗАГОЛОВОК ВТОРОГО УРОВНЯ)